# DETR-Based Object Detection: Data Preprocessing & DataLoader Pipeline

This document details the complete process undertaken to prepare the BDD100K dataset for object detection training using a DETR-like architecture. The workflow includes acquiring and filtering the data, converting annotations, balancing the dataset, augmenting underrepresented weather conditions, and building an efficient PyTorch-compatible data loading pipeline. The goal was to ensure the model can generalize well across diverse weather conditions while maintaining compatibility with transformer-based object detection models.

## Dataset Acquisition and Weather-Based Filtering

We began by downloading the BDD100K dataset from Kaggle because the original Berkeley-hosted download site was inaccessible. Once the dataset was obtained, we examined its metadata and discovered that it included weather annotations for each image. Our objective was to train a model that performs robustly across a variety of weather conditions, so we parsed the weather distribution and identified a class imbalance.

The dataset initially contained over 37,000 images labeled as 'clear', while 'foggy' images were extremely underrepresented with only about 130 samples. To ensure our model would not overfit on dominant weather conditions like 'clear' and would generalize well to rare adverse conditions, we decided to create a custom subset. We selected 5,000 images each from 'clear', 'overcast', and 'partly cloudy' weather categories. All available samples from 'rainy', 'snowy', and 'foggy' conditions were also included. This resulted in a balanced and diverse training set of over 25,000 images.

## Converting Annotations to COCO Format

The original BDD100K annotations were provided in Supervisely JSON format, which is incompatible with the DETR architecture. DETR expects annotations in the COCO format, which includes standardized fields for images, annotations, and categories. We wrote a custom script that parsed the original annotation files, filtered for our selected weather categories, and converted the bounding box annotations into the `[x, y, width, height]` format required by COCO.

In addition to bounding boxes, the script ensured that each image entry was matched correctly with its annotations and weather category. Invalid boxes (e.g., width or height < 1) were discarded to avoid training instability. Each image’s metadata and category mappings were stored in a new JSON annotation file. This conversion was applied separately for training, validation, and test splits.

# Splitting Dataset by Weather Condition

While the training set was combined into a single balanced pool, we retained weather-based folders for validation and testing. This was done so that we could later evaluate the performance of the model under specific weather conditions (e.g., mAP on foggy vs clear images). The directory structure was organized as follows:

```
dataset/
├── train/
│   ├── images/
│   └── train_annotations.json
├── val/
│   ├── clear/images/
│   ├── foggy/images/
│   └── coco_annotations.json
├── test/
│   └── [same structure as val/]
```

# Addressing Data Imbalance with Synthetic Fog Images

Due to the extremely low count of foggy images (only 130), we needed to artificially augment this class. First, we attempted Albumentations' built-in `RandomFog` transformation, but its visual output was subtle and insufficient. We then implemented a custom OpenCV-based fog synthesis function that better emulated real-world haze. Using this, we generated synthetic foggy versions of clear-weather images to improve representation.

The foggy samples were saved in a dedicated folder and appended to the training dataset. Care was taken to preserve the annotations for these images by copying the bounding boxes from the original clear images.

# Mapping Category IDs Between BDD100K and COCO

The BDD100K dataset defines categories that do not directly match the COCO categories used to train DETR. For example, 'rider' and 'traffic sign' exist in BDD100K but not in COCO. Therefore, we filtered the category list and retained only those compatible with COCO and DETR's pretrained model:

- person → person
- bike → bicycle

- car → car
- motor → motorcycle
- bus → bus
- truck → truck
- traffic light → traffic light

All other labels were excluded to maintain consistency. A mapping dictionary was created and used during inference and training to align category indices.

## Designing the PyTorch DataLoader Pipeline

To efficiently load and preprocess data during training, we implemented a custom PyTorch `Dataset` class. This class loads image-annotation pairs using `pycocotools`, applies on-the-fly transformations using Albumentations, and returns batched PyTorch tensors ready for training.

Each image is transformed through a sequence of steps:

1. Resized to a maximum of 800 pixels while preserving aspect ratio
2. Padded to a square size (800x800)
3. Randomly flipped horizontally with a 50% chance
4. Randomly brightened or contrasted
5. Optionally blurred for slight realism
6. Normalized to ImageNet standards
7. Converted to PyTorch tensors

Bounding boxes are resized in sync with their corresponding images using Albumentations' built-in support for bounding box transformations.

An accompanying `collate_fn` was implemented to allow batch processing of images with varying numbers of annotations. This function simply groups image and target dictionaries into lists for batch processing.

## Output Format for Model Consumption

After passing through the DataLoader, the output is a tuple of:

- `List[Tensor]`: each tensor is an image of shape `(3, H, W)`
- `List[Dict]`: each dict contains keys `boxes`, `labels`, and `image_id`

This format is compatible with detection models such as DETR, Faster R-CNN, and custom transformer-based heads.

## Summary

The data preprocessing workflow allowed us to transform the BDD100K dataset into a training-ready, COCO-compatible format. We addressed dataset imbalance by carefully sampling and generating synthetic foggy data. A custom PyTorch pipeline was built using Albumentations to handle all image resizing, normalization, and augmentation needs. This setup ensures that our DETR-like model receives clean, balanced, and dynamically transformed data during training, enabling robust detection across weather conditions.